

eada business school barcelona

Data Preparation for Machine Learning Projects

Recap from Previous Sessions

- **Computer Vision** teaches machines to **recover scene meaning** from **images** — not just measure light.
- **Images** can form **from** many kinds of **energy**, not only visible light.
- Digital images are arrays of pixels; **a single pixel rarely carries meaning without context.**
- CV tasks range from **classification** to **detection** to **segmentation**.
- **MILK10k** is a real dataset: 10,480 images, 5,240 lesions, 11 diagnosis classes, and severe class imbalance.

Recap from Previous Sessions

- An image is a **matrix of pixels**, formally the function F(X, Y); its amplitude is the pixel's **intensity**.
- **Resolution** and **bit/color depth** determine how much spatial and tonal detail an image captures; **sampling/quantization** are the two approximations behind them.
- **RGB** images stack three 0–255 channels into a **H × W × 3** array.
- **Grayscale** collapses the channels into one channel, at the cost of color information.
- **Histograms** summarize tonal distribution.
- **Normalization** rescales pixel values so neural networks train faster and more reliably (*keep gradients stable, equalize feature contribution, compatibility with pretrained models, numerical stability).*

SESSION 3

Learning Objectives

- **Explain** why data preparation, not model choice, is usually what makes or breaks a real project.
- **Design** a leak-free train / validation / test split, grouped at the correct unit.
- **Diagnose** class imbalance quantitatively and choose an appropriate strategy to handle it.
- **Build** a data augmentation pipeline appropriate for medical images.
- **Implement** a reusable, robust MILK10k data-loading pipeline in Python that later sessions build on.

SESSION 3

Today's Agenda

PART 1 — THEORY	BREAK	PART 2 — HANDS-ON PYTHON
80 min	20 min	80 min

- **Part 1:** splitting, leakage, class imbalance, augmentation, robust pipelines.
- **Break**: 20 minutes.
- **Part 2:** build a grouped, leak-free split of MILK10k, quantify imbalance, build and visualize an augmentation pipeline, and wrap it all in a reusable Dataset class.

SESSION 3

Part 1 Outline — Theory

- Why **data preparation** decides project success, more often than model architecture.
- **Train / validation / test** splitting.
- **Class imbalance**: measuring it, and MILK10k's real numbers.
- **Data leakage**: what it is, common forms, and a real case study.
- **Data augmentation**: what to use, and what to avoid for medical images.

SESSION 3

Part 1 — Theory

Why data quality decides model quality

eada business school barcelona

Why Data Preparation Matters

? *Which decides a project's success more – the model architecture, or the data it's trained on?*

Why Data Preparation Matters

? *Which decides a project's success more – the model architecture, or the data it's trained on?*

- In industry surveys, data scientists consistently report spending **60–80% of project time** on data work — not modeling.
- **Garbage in, garbage out**: a state-of-the-art architecture trained on leaked, mislabeled, or wildly imbalanced data will still fail in production.
- Worse — a data problem often produces a model that looks great on your test metric and then fails silently in the real world.
- Session 1 introduced train/val/test splits and the generalization gap conceptually. Today we make that rigorous and MILK10k-specific.

Recap — MILK10k, By the Numbers

- **10,480 images**, covering **5,240 unique lesions** — exactly 2 images per lesion (one dermoscopic, one clinical close-up).
- Every lesion has a stable **lesion_id** (e.g. `IL_1612768`) linking its two images together.
- Each lesion carries `diagnosis_1` (Benign / Malignant / Indeterminate), a `diagnosis_2/3/4` hierarchy, demographic fields, and a `copyright_license`.
- This structure — multiple images per subject, richly labeled — is typical of real medical-imaging datasets, and is exactly what makes splitting non-trivial.

Recap — MILK10k, By the Numbers

- **10,480 images**, covering **5,240 unique lesions** — exactly 2 images per lesion (one dermoscopic, one clinical close-up).
- Every lesion has a stable **lesion_id** (e.g. `IL_1612768`) linking its two images together.
- Each lesion carries `diagnosis_1` (Benign / Malignant / Indeterminate), a `diagnosis_2/3/4` hierarchy, demographic fields, and a `copyright_license`.
- This structure — multiple images per subject, richly labeled — is typical of real medical-imaging datasets, and is exactly what makes splitting non-trivial.

Train / Validation / Test — A Deeper Look

Three splits, three distinct jobs:

Split	Used for	Touched how often?
Train		
Validation		
Test		

If you tune anything based on the test set — even 'just checking' — it quietly becomes a second validation set, and your final number stops being trustworthy.

Train / Validation / Test — A Deeper Look

Three splits, three distinct jobs:

Split	Used for	Touched how often?
Train	Fitting model parameters (weights)	Every training step
Validation		
Test		

Train / Validation / Test — A Deeper Look

Three splits, three distinct jobs:

Split	Used for	Touched how often?
Train	Fitting model parameters (weights)	Every training step
Validation	Tuning hyperparameters, choosing epochs/checkpoints, comparing models	Every epoch or so
Test		

Train / Validation / Test — A Deeper Look

Three splits, three distinct jobs:

Split	Used for	Touched how often?
Train	Fitting model parameters (weights)	Every training step
Validation	Tuning hyperparameters, choosing epochs/checkpoints, comparing models	Every epoch or so
Test	One final, honest performance estimate	Exactly once, at the very end

Train / Validation / Test — A Deeper Look

Three splits, three distinct jobs:

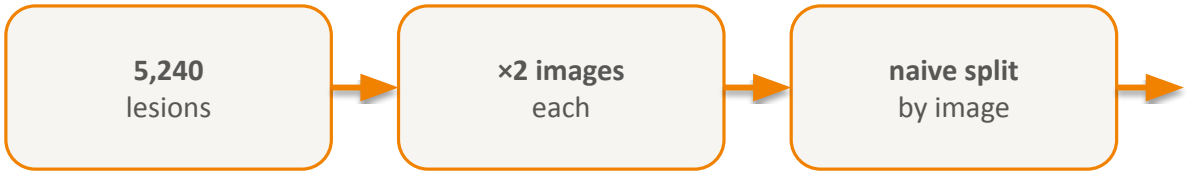
Split	Used for	Touched how often?
Train	Fitting model parameters (weights)	Every training step
Validation	Tuning hyperparameters, choosing epochs/checkpoints, comparing models	Every epoch or so
Test	One final, honest performance estimate	Exactly once, at the very end

If you tune anything based on the test set — even 'just checking' — it quietly becomes a second validation set, and your final number stops being trustworthy.

The Group-Leakage Trap

? *MILK10k gives 2 images per lesion. What could go wrong if you split image-by-image?*

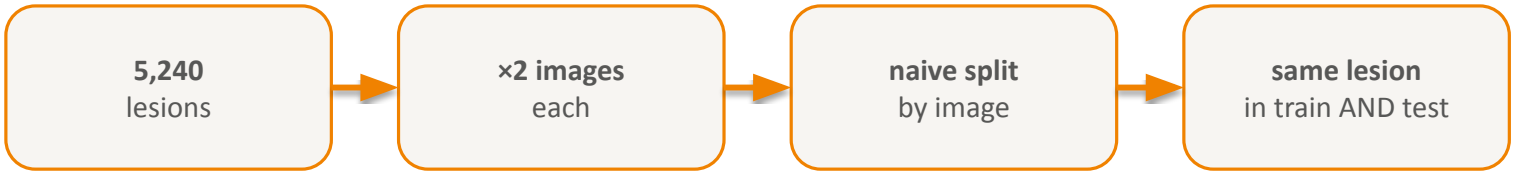
MILK10k gives 2 images per lesion. Naively splitting by image is dangerous:



The Group-Leakage Trap

? *MILK10k gives 2 images per lesion. What could go wrong if you split image-by-image?*

MILK10k gives 2 images per lesion. Naively splitting by image is dangerous:



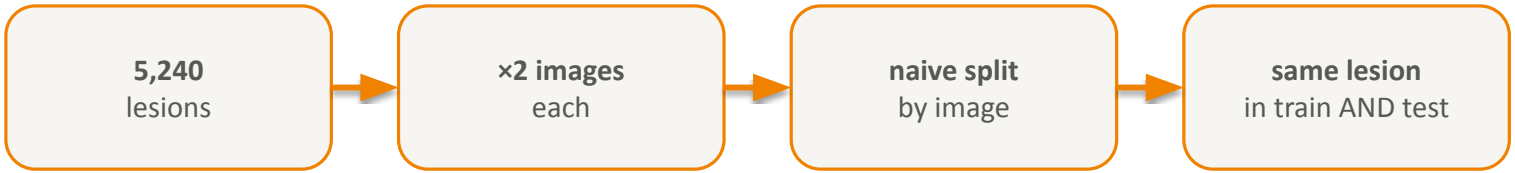
KEY IDEA

Always split by the **grouping unit that matters** — here, `lesion_id` — never by raw row. `sklearn.model_selection.GroupShuffleSplit` and `GroupKFold` exist for exactly this reason.

The Group-Leakage Trap

? *MILK10k gives 2 images per lesion. What could go wrong if you split image-by-image?*

MILK10k gives 2 images per lesion. Naively splitting by image is dangerous:



KEY IDEA

Always split by the **grouping unit that matters** — here, `lesion_id` — never by raw row. `sklearn.model_selection.GroupShuffleSplit` and `GroupKFold` exist for exactly this reason.

CODE

MILK10k Class Distribution (Real Numbers)

Per-lesion diagnosis counts, from training_gt.csv:

Code	Meaning	Lesions	% of total
BCC	Basal cell carcinoma	2,522	48.1%
NV	Nevus	746	14.2%
BKL	Benign keratosis-like lesion	544	10.4%
SCCKA	Squamous cell carcinoma / keratoacanthoma	473	9.0%
MEL	Melanoma	450	8.6%
AKIEC	Actinic keratosis / intraepithelial carcinoma	303	5.8%
DF	Dermatofibroma	52	1.0%

4 more rare classes (DF, INF, VASC, BEN_OTH, MAL_OTH) together make up under 4% of lesions — some with fewer than 10 examples.

CODE

Why Is MILK10k So Imbalanced?

? *69% of MILK10k's lesions are malignant. Does that mean 69% of real-world skin lesions are?*

CODE

KEY IDEA

Always ask **how a dataset was collected**, not just what it contains.
Collection process shapes the label distribution and your model's blind spots.

SESSION 3 · THEORY

Why Is MILK10k So Imbalanced?

? *69% of MILK10k's lesions are malignant. Does that mean 69% of real-world skin lesions are?*

- It is **not** a random sample of skin lesions seen in clinics — **69.3%** of lesions here are labeled Malignant.
- That is a **sampling bias**, not a fact about the world: MILK10k, like most public dermoscopy datasets, is enriched with cases that were sent for **biopsy** (`diagnosis_confirm_type = histopathology`, 95.7% of images).
- Doctors biopsy lesions they are **worried about** — so malignant and borderline cases are systematically over-represented versus the general population.
- This has a direct project consequence: a model trained here should **not** be read as estimating real-world malignancy prevalence — only as distinguishing diagnosis **given** that a lesion was already flagged for biopsy.

KEY IDEA

Always ask **how a dataset was collected**, not just what it contains.
Collection process shapes the label distribution and your model's blind spots.

Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy		
Precision / Recall		
Macro-F1		
ROC-AUC / PR-AUC		

Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy	% correct overall	A model predicting 'BCC' always would score ~48%
Precision / Recall		
Macro-F1		
ROC-AUC / PR-AUC		

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy	% correct overall	A model predicting 'BCC' always would score ~48%
Precision / Recall	Correctness / completeness per class	Recall on MEL (melanoma) matters far more than recall on DF
Macro-F1		
ROC-AUC / PR-AUC		

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy	% correct overall	A model predicting 'BCC' always would score ~48%
Precision / Recall	Correctness / completeness per class	Recall on MEL (melanoma) matters far more than recall on DF
Macro-F1	F1 averaged equally across classes	Forces rare classes (VASC, DF) to count as much as BCC
ROC-AUC / PR-AUC		

F1 is calculated as follows:

$$F_1 = 2 * \frac{precision * recall}{precision + recall}$$

where:

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

In "macro" F1 a separate F1 score is calculated for each `species` value and then averaged.

Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy	% correct overall	A model predicting 'BCC' always would score ~48%
Precision / Recall	Correctness / completeness per class	Recall on MEL (melanoma) matters far more than recall on DF
Macro-F1	F1 averaged equally across classes	Forces rare classes (VASC, DF) to count as much as BCC
ROC-AUC / PR-AUC	Ranking quality across thresholds	Threshold-independent; PR-AUC is more informative than ROC-AUC under imbalance

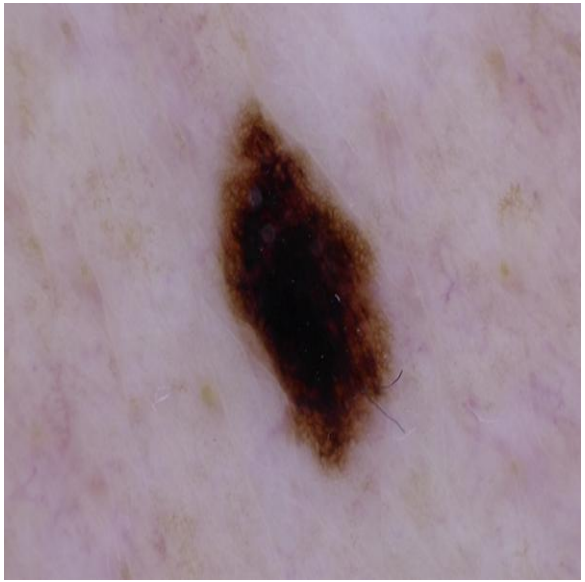
Metrics That Survive Imbalance

With classes this skewed, accuracy is close to meaningless:

Metric	What it measures	Why it matters here
Accuracy	% correct overall	A model predicting 'BCC' always would score ~48%
Precision / Recall	Correctness / completeness per class	Recall on MEL (melanoma) matters far more than recall on DF
Macro-F1	F1 averaged equally across classes	Forces rare classes (VASC, DF) to count as much as BCC
ROC-AUC / PR-AUC	Ranking quality across thresholds	Threshold-independent; PR-AUC is more informative than ROC-AUC under imbalance

The Cost of Errors Is Not Symmetric

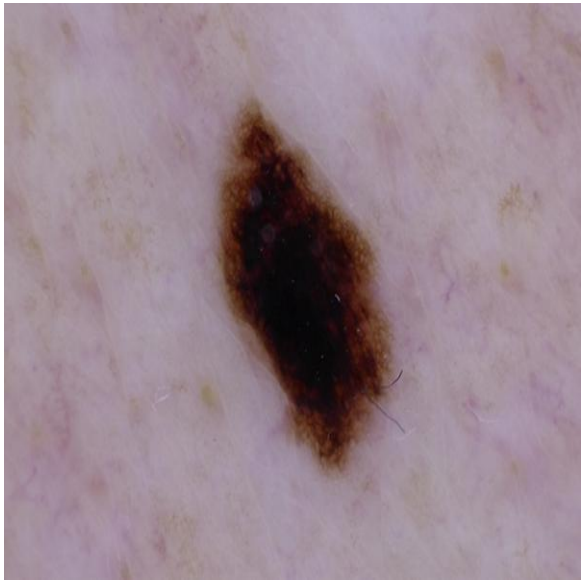
- A **false negative** on a melanoma
- A **false positive** on a benign nevus



Melanoma in situ — malignant · age 85

The Cost of Errors Is Not Symmetric

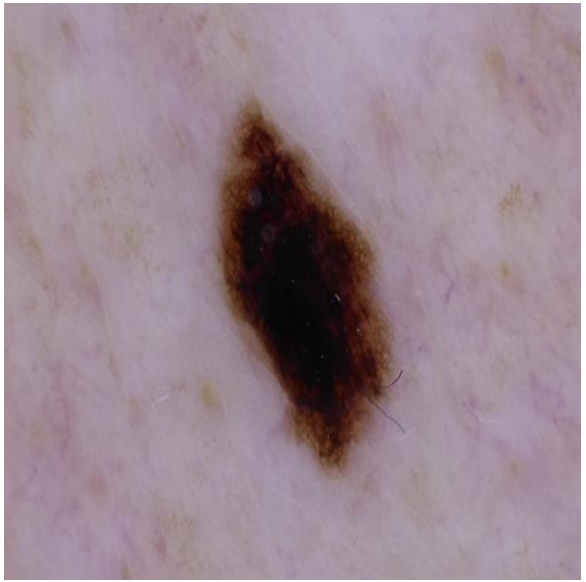
- A **false negative** on a melanoma (calling it benign) can cost a life.
- A **false positive** on a benign nevus (flagging it malignant) costs an unnecessary biopsy — unpleasant, but not dangerous.
- This asymmetry means the **right** operating point is rarely 'maximize accuracy' — it is a **clinical decision**, made with a domain expert, about acceptable false-negative rates.



Melanoma in situ — malignant · age 85

The Cost of Errors Is Not Symmetric

- A **false negative** on a melanoma (calling it benign) can cost a life.
- A **false positive** on a benign nevus (flagging it malignant) costs an unnecessary biopsy — unpleasant, but not dangerous.
- This asymmetry means the **right** operating point is rarely 'maximize accuracy' — it is a **clinical decision**, made with a domain expert, about acceptable false-negative rates.



Melanoma in situ — malignant · age 85

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights		
Oversampling		
Undersampling		
Targeted augmentation		
Stratified splitting		

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights	Penalize errors on rare classes more in the loss	Can destabilize training if weights are extreme
Oversampling		
Undersampling		
Targeted augmentation		
Stratified splitting		

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights	Penalize errors on rare classes more in the loss	Can destabilize training if weights are extreme
Oversampling	Repeat/resample rare-class examples per epoch	Risk of overfitting to repeated examples
Undersampling		
Targeted augmentation		
Stratified splitting		

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights	Penalize errors on rare classes more in the loss	Can destabilize training if weights are extreme
Oversampling	Repeat/resample rare-class examples per epoch	Risk of overfitting to repeated examples
Undersampling	Drop some majority-class examples	Throws away data — rarely ideal with already-small datasets
Targeted augmentation		
Stratified splitting		

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights	Penalize errors on rare classes more in the loss	Can destabilize training if weights are extreme
Oversampling	Repeat/resample rare-class examples per epoch	Risk of overfitting to repeated examples
Undersampling	Drop some majority-class examples	Throws away data — rarely ideal with already-small datasets
Targeted augmentation	Augment rare classes more aggressively	Must still respect diagnostic validity
Stratified splitting		

Strategies for Class Imbalance

No single fix is always right — these are commonly combined:

Strategy	Idea	Watch out for
Class weights	Penalize errors on rare classes more in the loss	Can destabilize training if weights are extreme
Oversampling	Repeat/resample rare-class examples per epoch	Risk of overfitting to repeated examples
Undersampling	Drop some majority-class examples	Throws away data — rarely ideal with already-small datasets
Targeted augmentation	Augment rare classes more aggressively	Must still respect diagnostic validity
Stratified splitting	Preserve class ratios in every split	Doesn't fix imbalance — just prevents it from worsening by chance

Stratified, Grouped Splitting

? *Splitting needs to respect grouping AND class balance at once. Can one scikit-learn call do both?*

MILK10k splitting needs both properties at once:



o does exactly this — grouped **and** stratified in one call.

Stratified, Grouped Splitting

? *Splitting needs to respect grouping AND class balance at once. Can one scikit-learn call do both?*

MILK10k splitting needs both properties at once:



- `StratifiedGroupKFold` does exactly this — grouped **and** stratified in one call.

Cross-Validation for Rare Classes

- With classes like **DF** (52 lesions) or **MAL_OTH** (9 lesions), a single random split can put *zero* examples of a class into validation — you would have no way to measure that class's performance.
- **k-fold cross-validation** rotates which fold is held out, so every lesion is validated on exactly once across k runs — much more robust with small classes.
- For MILK10k specifically: use **StratifiedGroupKFold(n_splits=N)** — e.g. N=5 folds balances compute cost against a stable per-class estimate.
- For classes under ~10 examples (MAL_OTH), even 5-fold CV gives unstable estimates — report those numbers with appropriate caution, or group them into a broader 'other' category for training.

Cross-Validation for Rare Classes

- With classes like **DF** (52 lesions) or **MAL_OTH** (9 lesions), a single random split can put *zero* examples of a class into validation — you would have no way to measure that class's performance.
- **k-fold cross-validation** rotates which fold is held out, so every lesion is validated on exactly once across k runs — much more robust with small classes.
- For MILK10k specifically: use **StratifiedGroupKFold(n_splits=N)** — e.g. N=5 folds balances compute cost against a stable per-class estimate.
- For classes under ~10 examples (MAL_OTH), even 5-fold CV gives unstable estimates — report those numbers with appropriate caution, or group them into a broader 'other' category for training.

CODE

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	
Preprocessing leakage	
Duplicate leakage	
Target leakage	

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	Same lesion_id's two images split across train and test
Preprocessing leakage	
Duplicate leakage	
Target leakage	

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	Same lesion_id's two images split across train and test
Preprocessing leakage	Normalizing using statistics (mean/std) computed on the *full* dataset, test included
Duplicate leakage	
Target leakage	

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	Same lesion_id's two images split across train and test
Preprocessing leakage	Normalizing using statistics (mean/std) computed on the *full* dataset, test included
Duplicate leakage	Near-duplicate images (re-photographs of the same lesion) landing on both sides
Target leakage	

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	Same lesion_id's two images split across train and test
Preprocessing leakage	Normalizing using statistics (mean/std) computed on the *full* dataset, test included
Duplicate leakage	Near-duplicate images (re-photographs of the same lesion) landing on both sides
Target leakage	A feature that encodes the label indirectly, e.g. `diagnosis_confirm_type` correlating with `diagnosis_1`

What Is Data Leakage?

? *A model scores suspiciously well on the test set. Is that always good news?*

Leakage = information from outside the training data (often, indirectly, from the test set) illegitimately influencing the model:

Type	Example in MILK10k
Group leakage	Same lesion_id's two images split across train and test
Preprocessing leakage	Normalizing using statistics (mean/std) computed on the *full* dataset, test included
Duplicate leakage	Near-duplicate images (re-photographs of the same lesion) landing on both sides
Target leakage	A feature that encodes the label indirectly, e.g. `diagnosis_confirm_type` correlating with `diagnosis_1`

CODE *(pd.crosstab → frequency counts (how many times each combination of values appears)*

Case Study — Leakage in Medical Imaging

- Leakage is not a toy concern: multiple published medical-imaging papers have been shown to report inflated accuracy due to patient-level leakage between splits.

- The MILK10k analogue: `image_manipulation` ("instrument only" vs "altered") or visible skin-marking pen ink could correlate with which clinic contributed the image — and thus, indirectly, with diagnosis rates at that clinic.

CODE *(pd.crosstab → frequency counts (how many times each combination of values appears)*

Case Study — Leakage in Medical Imaging

- Leakage is not a toy concern: multiple published medical-imaging papers have been shown to report inflated accuracy due to patient-level leakage between splits.

- The MILK10k analogue: `image_manipulation` ("instrument only" vs "altered") or visible skin-marking pen ink could correlate with which clinic contributed the image — and thus, indirectly, with diagnosis rates at that clinic.

KEY IDEA

A model with suspiciously high accuracy should make you **more** suspicious of leakage, not less.

Data Quality Checks Before You Split

Real numbers from metadata.csv — always audit missingness first:

Column	Action
age_approx	Impute with median, or drop rows
anatom_site_general	Keep as explicit 'unknown' category
diagnosis_4	Expected — most lesions stop at diagnosis_2/3
melanocytic	NaN means 'not applicable', not missing

Rule of thumb: a NaN is only a bug if the field should always be populated.
Check the data dictionary before 'fixing' anything.

Data Quality Checks Before You Split

Real numbers from metadata.csv — always audit missingness first:

Column	Action
age_approx	Impute with median, or drop rows
anatom_site_general	Keep as explicit 'unknown' category
diagnosis_4	Expected — most lesions stop at diagnosis_2/3
melanocytic	NaN means 'not applicable', not missing

Rule of thumb: a NaN is only a bug if the field should always be populated.
Check the data dictionary before 'fixing' anything.

CODE

Data Augmentation — Why and What

? *If you can't collect more training images, can you still make your model see more variety?*

Data Augmentation — Why and What

? *If you can't collect more training images, can you still make your model see more variety?*

Augmentation expands effective training data by applying label-preserving transforms:

Data Augmentation — Why and What

? *If you can't collect more training images, can you still make your model see more variety?*

Augmentation expands effective training data by applying label-preserving transforms:

Flip mirror	**Rotate** small angle	**Brightness** / contrast	**Crop** / zoom

Data Augmentation — Why and What

? *If you can't collect more training images, can you still make your model see more variety?*

Augmentation expands effective training data by applying label-preserving transforms:

Flip mirror	**Rotate** small angle	**Brightness** / contrast	**Crop** / zoom

- Applied **only** to the training set, generated fresh (or cached) each epoch — never to validation or test.

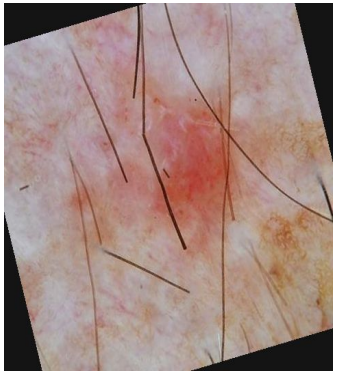
Real Augmentations on a MILK10k Image



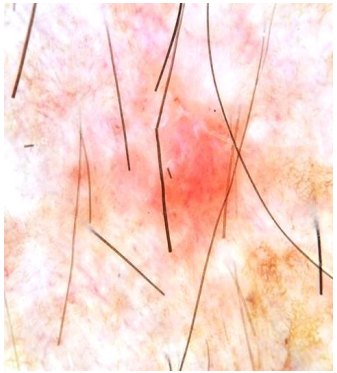
original



horizontal flip



rotate 15°



brightness ×1.45



zoom crop

Augmentation Cautions for Medical Images

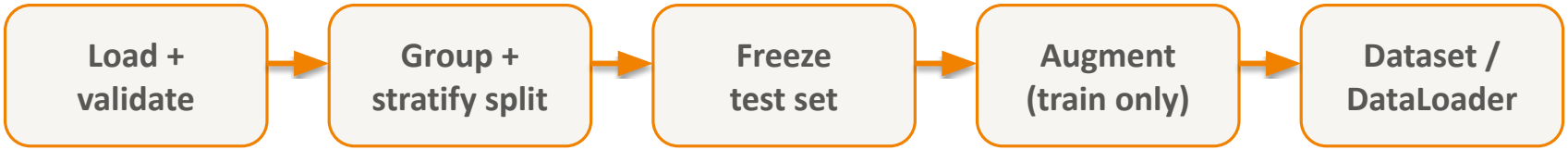
- Not every augmentation is safe just because it works for natural photos (cats, cars, street scenes).
- **Vertical flips** are usually fine for skin lesions (no canonical 'up'), but would be wrong for, say, X-rays where anatomy has a fixed orientation.
- **Aggressive color jitter** is risky here: erythema (redness) and pigmentation are diagnostically meaningful — over-jittering hue can turn a benign nevus into something that **looks** like it has abnormal pigmentation.
- **Cropping** must not remove the lesion itself — random crops need a minimum-overlap constraint with the lesion's location.

KEY IDEA

Every augmentation is an implicit claim: **"this transform never changes the true label."**
For medical images, that claim needs a domain expert's sign-off, not just intuition.

Building a Robust Pipeline

Putting every idea from today into one ordered pipeline:



Part 1 Recap — Key Takeaways

- Split on the **grouping unit that matters** (`lesion_id`), not the raw image row — otherwise you leak.
- MILK10k is imbalanced **by construction** (biopsy-enriched sampling) — accuracy alone will mislead you.
- Use **StratifiedGroupKFold** to get splits that are simultaneously leak-free and keep the class proportion.
- Data leakage is often silent — a suspiciously good result is a reason to **audit**, not celebrate.
- Augmentation must respect the medical meaning of the image, not just be 'whatever torchvision offers by default'.

TIME FOR A BREAK

20 Minutes

Stretch, grab a coffee, and be back on time.

eada
business school
barcelona

SESSION 3

Part 2 — Hands-on Python

Building a leak-free MILK10k pipeline

eada business school barcelona

Coming Up Next & References

- **Next — Session 4: Filters & Convolution.** We move from *preparing* pixels to *transforming* them: blur, sharpen and edge-detection filters, and the convolution operation that both classical vision and every CNN are built on.